Tokenizer or Training Data? How a Small Uzbek Language Model Beats Much Larger Multilingual Models

Islombek Turdiyev · Independent project, 2025–2026 · Code, data and full paper: github.com/TurdiyevIslombek/uzbek-gpt-from-scratch · Models: huggingface.co/IslombekT · Demo: generate.py in the repository writes Uzbek text from a prompt

The problem. Language models cut Uzbek, spoken by about 40 million people, into far more pieces (tokens) than English, which makes them slower and more expensive for Uzbek speakers. **My question:** for a small language like Uzbek, what matters more: a tokenizer built for the language, or training the model on the language from scratch? To my knowledge no one had measured this for Uzbek, so I built both and separated the two effects by experiment.

What I built

- **A tokenizer** for Uzbek (16,384 tokens, byte-level BPE) with rules that unify the many ways people type the apostrophe in *o‘* and *g‘*.
- **A 103-million-parameter language model**, trained from scratch on 1.06 billion Uzbek tokens: 3.4 hours on one rented GPU, about $3.60.
- **A control model:** the same architecture trained on exactly the same documents, but with mGPT’s multilingual tokenizer, to isolate what the tokenizer does. About 51 hours on free Kaggle GPUs.
- **Opponents:** mGPT-1.3B, used as-is and fine-tuned on Uzbek; four current Qwen models of 0.5–1.7 billion parameters.

Results

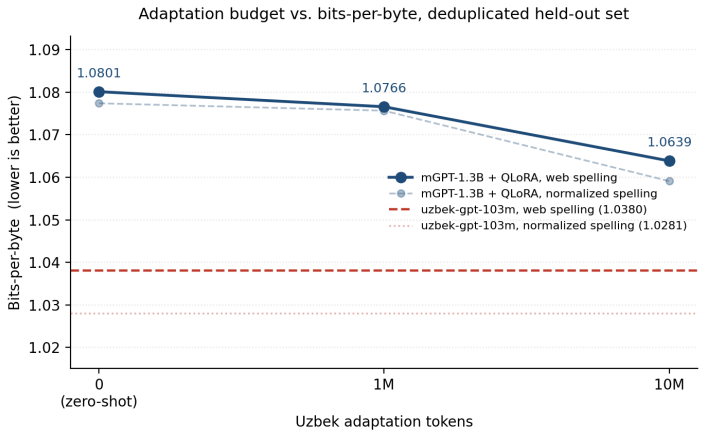
Bits-per-byte measures how well a model predicts unseen text, fairly across different tokenizers (lower is better). 474 held-out documents in their original web spelling.

Model	Size	Bits/byte
My model (from scratch, my tokenizer)	**0.10B**	**1.038**
Control: same model, mGPT’s tokenizer	0.23B	1.031
mGPT, fine-tuned on Uzbek	1.3B	1.077
mGPT, as-is	1.3B	1.080
Qwen3 (best of four current models)	1.7B	1.532

95% confidence intervals from a paired bootstrap over the 474 documents (10,000 resamples) exclude zero for every difference shown.

1. **The small models win.** Mine needs about 4% fewer bits than fine-tuned mGPT (13× larger) and 32% fewer than Qwen3 (17× larger).
2. **Quality comes from training data, not the tokenizer.** The control model reaches comparable quality: 0.007 better than mine on web spelling, 0.005 worse on normalized spelling. Both beat mGPT because they trained on a billion Uzbek tokens.
3. **The tokenizer pays off in efficiency.** It uses 1.749 tokens per word, the fewest of 11 tested: about half of GPT-2 and 45% fewer than mGPT’s. My model reached comparable quality with less than half the parameters and about a third of the training compute.

Can fine-tuning catch up?



Fine-tuning mGPT on 10× more Uzbek text narrows the gap but does not close it (still 0.026 behind). A rough extrapolation from the 1M and 10M runs suggests catching up could take on the order of 1 billion tokens, about as much as training from scratch.

How I tested my own work

Before submitting, I checked whether my experiments really were what I thought. Four were not, and each problem made my results look better than they were:

1. **My test set was inside my training data.** 11 of 12 test passages were in the training file. I rebuilt it by fingerprinting 27.4 million passages of training text.
2. **39% of my validation data was duplicated** from training data, because web pages repeat. I removed every overlapping document (306 of 780).
3. **My test text was spelled the way my own tokenizer spells**, which favored my model. I recovered the original web text of all 474 documents and re-scored every model.
4. **My control model had not run as I described it.** It read the test documents and stopped halfway through training. I retrained it on exactly my model’s documents.

Each correction made my claim smaller. The tokenizer’s share of my quality advantage went from “all of it” to none. I published the corrected numbers, and the corrected code reproduces my original ones exactly, so every change can be checked.

What I take from it: for a small language, quality comes from text in that language; a tokenizer built for it makes the model smaller, cheaper and faster.
AI assistance: I used Claude (Anthropic) to learn methods, write and debug code, and edit writing. It also flagged problems 1 and 4 above, which I confirmed. I ran every check, trained every model and ran every experiment myself.